



CSE 402

University of Asia Pacific

Course Name : Operating System Lab

Lab Report 1

Topic : FSFC Algorithm

Presented by,

Md Mashrur Rahman Masfi
Reg : 23101182
Section : D1

Presented to,

Atia Rahman Orthi
Lecturer, UAP

Problem Statement :

In a multiprogramming environment the CPU must constantly decide which of the waiting processes should execute next. This experiment implements the First Come First Serve (FCFS) policy, where processes are dispatched strictly in the order they enter the ready queue, and a running process keeps the CPU until its burst finishes (non-preemptive scheduling).

The task is to write a Python program that accepts five processes, each described by a Process ID, Arrival Time and Burst Time, and then:

- Arranges the processes in increasing order of arrival time.
- Simulates the non-preemptive execution on a single CPU to obtain each process's Completion Time (CT).
- Derives the Turnaround Time ($TAT = CT - AT$) and Waiting Time ($WT = TAT - BT$).
- Computes the average turnaround time and average waiting time.
- Prints the execution timeline (Gantt chart) so the result can be verified by hand.

Source Code :

```

1  cpu_schedule = [(3, 3), # P1
2      (2, 1), # P2
3      (5, 2), # P3
4      (0, 3), # P4
5      (1, 2)] # P5
6
7
8  def fcfs(cpu_schedule):
9
10     procs = [(f"P{i+1}", at, bt) for i, (at, bt) in enumerate(cpu_schedule)]
11
12
13     order = sorted(procs, key=lambda p: (p[1], p[0]))
14
15     time, results, gantt = 0, [], []
16
17     for pid, at, bt in order:
18
19         if time < at:
20             gantt.append(("IDLE", time, at))
21             time = at
22
23         start = time
24         ct = start + bt # Completion Time
25         tat = ct - at # Turnaround Time
26         wt = tat - bt # Waiting Time
27
28         gantt.append((pid, start, ct))
29         results.append([pid, at, bt, ct, tat, wt])
30         time = ct
31
32     return results, gantt
33
34
35 def print_results(results):
36     print(f'{"PID":<5}{"AT":>3}{"BT":>4}{"CT":>4}{"TAT":>5}{"WT":>4}')
37     print("-" * 25)
38     for pid, at, bt, ct, tat, wt in sorted(results):
39         print(f'{"pid":<5}{"at":>3}{"bt":>4}{"ct":>4}{"tat":>5}{"wt":>4}')
40
41     n = len(results)
42     print("-" * 25)
43     print(f"Average Turnaround Time = {sum(r[4] for r in results) / n:.2f}")
44     print(f"Average Waiting Time = {sum(r[5] for r in results) / n:.2f}")
45
46
47 def print_gantt(gantt):
48     print("\nGantt Chart:")
49     top, bottom = "", ""
50     for pid, s, e in gantt:
51         top += f"| {pid:^4} "
52         bottom += f"{s:<7}"
53     top += "|"
54     bottom += str(gantt[-1][2])
55     print(top)

```

Output Screenshot :

```
masfirahman@Masfis-MacBook-Pro CSE-402-Operating-System-Lab- % /usr/local/bin/python3 /Users/masfirahman/CSE-402-Operating-System-Lab-/FCFS.py
PID  AT  BT  CT  TAT  WT
-----
P1   3   3   9   6   3
P2   2   1   6   4   3
P3   5   2  11   6   4
P4   0   3   3   3   0
P5   1   2   5   4   2

Average Turnaround Time = 4.60
Average Waiting Time    = 2.40

Gantt Chart:
| P4 | P5 | P2 | P1 | P3 |
0   3   5   6   9  11
```

Discussion :

The aim of this experiment was to implement and verify First-Come-First-Serve (FCFS) scheduling on a small five-process workload. As expected from a non-preemptive FIFO policy, the dispatch order followed arrival order, not process numbering: P4 (AT = 0) began immediately, and each subsequent CPU assignment went to the process that had waited longest in the ready queue - P5, then P2, P1, and finally P3. The resulting Gantt timeline (0 → 3 → 5 → 6 → 9 → 11) contains no idle gaps, because at every completion instant at least one process was already waiting. Consequently, the overall finish time of 11 units is exactly the sum of the five bursts (3 + 2 + 1 + 3 + 2 = 11). The computed metrics satisfy the standard identities $TAT = CT - AT$ and $WT = TAT - BT$ for every process. Taking P3 as an example: it arrives at 5 and completes at 11, giving a turnaround of 6; subtracting its 2-unit burst leaves a waiting time of 4. Aggregating over all five processes yields an average turnaround time of 4.60 and an average waiting time of 2.40, in exact agreement with the hand-calculated table, which validates the simulation. At the same time, the trace clearly demonstrates FCFS's principal weakness: it considers when a process arrives but never how long it will run. P3, although one of the shortest jobs in the set, suffers the largest wait (4 units) simply because it arrived last, and P1 spends 3 units queued behind shorter jobs. Such convoy-like behaviour - short processes trapped behind longer ones - inflates average waiting time and makes pure FCFS unsuitable for interactive or response-sensitive systems. Even so, its simplicity, predictability, and freedom from starvation explain why the FCFS principle still appears as a building block inside more sophisticated policies such as multilevel queues.

Advantages observed :

- The entire scheduler reduces to a single FIFO queue, making implementation and testing straightforward.
- Execution order is fully predictable from arrival times, which simplifies debugging and manual verification.
- Service in arrival order guarantees every process eventually runs; starvation is impossible.
- Context switches occur only at termination or blocking, so scheduling overhead is minimal.

Disadvantages observed :

- Being non-preemptive, a single long CPU burst delays every process behind it.
- Average waiting time is rarely optimal and typically exceeds that of SJF, SRTF, or Round Robin.

- The convoy effect penalizes short jobs arriving behind long ones (seen here with P3).
- Response time is inadequate for time-sharing workloads. The policy offers no mechanism for priorities or deadlines.

Conclusion :

The FCFS program arranged all processes in order of their arrival time and simulated non-preemptive execution. The resulting completion, turnaround, and waiting times matched both the manual calculations and the Gantt chart, producing an average turnaround time of 4.60 and an average waiting time of 2.40. This exercise confirmed that FCFS is straightforward, deterministic, and starvation-free. However, it also highlighted the algorithm's major drawback: since burst time is completely disregarded, shorter processes can end up waiting excessively behind longer ones—a phenomenon known as the convoy effect. Because of this limitation, real-world operating systems seldom use pure FCFS. Nevertheless, its core first-come-first-served principle continues to serve as the building block for more advanced scheduling policies, such as multilevel feedback queues.

Github repository : <https://github.com/MasfiRahman/CSE-402-Operating-System-Lab->